SHERLOCK

Security Review For Superlend

Collaborative Audit Prepared For:	**Superlend**
Lead Security Expert(s):	**0x3b** **santipu_**
Date Audited:	**February 12 - February 20, 2026**

Introduction

This update extends the Loop Vault by integrating an additional flash loan venue via Morpho, expanding available liquidity sources and improving execution flexibility. The totalAssets() implementation has been optimized to reduce gas consumption and improve read efficiency. An ERC-4626–compliant supply module has been introduced to standardize vault interactions. A Merkle-based claim module has been added to enable claiming of rewards accrued by the vault.

Scope

Repository: Superlend/superloop-core-contracts

Audited Commit: e9543636f21cde9df0d4adb2c6903a867d875899

Final Commit: e9543636f21cde9df0d4adb2c6903a867d875899

Files:

- src/common/DataTypes.sol
- src/core/DepositManager/DepositManager.sol
- src/core/Superloop/Superloop.sol
- src/core/Superloop/SuperloopVault.sol
- src/modules/callback/MorphoCallbackHandler.sol
- src/modules/merkl/MerklModule.sol
- src/modules/morpho/MorphoFlashloanModule.sol
- src/modules/vault/VaultActionModule.sol
- src/modules/vault/VaultSupplyModule.sol
- src/modules/vault/VaultWithdrawModule.sol

Final Commit Hash

e9543636f21cde9df0d4adb2c6903a867d875899

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.

- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	0	3

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue L-1: Missing slippage protection in `VaultSupplyModule` and `VaultWithdrawModule` [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-02-superlend-feb-12th-2026/issues/11

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

`VaultSupplyModule` and `VaultWithdrawModule` allow the Superloop operator to deposit into and withdraw from arbitrary external ERC4626 vaults. However, both modules execute `deposit()` / `withdraw()` without any form of slippage protection (i.e., no minimum shares received on `deposit`, and no maximum shares burned on `withdraw`). If the external vault's share price changes between when the operator prepares the transaction and when it is executed, the outcome can be materially worse than expected.

Both modules rely on the external vault's current conversion rate at execution time:

- **Supply path** approves an amount of underlying and calls `IERC4626(vault).deposit(amount, address(this))`. The returned shares can be lower than expected if the vault's share price has recently increased.
- **Withdraw path** determines amount and calls `IERC4626(vault).withdraw(amount, address(this), address(this))`. The call burns whatever number of shares the vault requires at execution time; this can be higher than expected if the share price has recently decreased.

This becomes problematic for vaults whose share price is volatile (e.g., vaults whose accounting depends on external oracle prices, on-chain liquidity conditions, or rapidly changing asset compositions). In those cases, a sudden share-price movement can cause the operator to receive fewer shares on deposit or spend more shares on withdrawal than intended, with no ability to enforce execution bounds.

Example (deposit):

1. Operator intends to deposit 1 WBTC into an external ERC4626 vault.
2. At the time the transaction is constructed, the vault's effective exchange rate implies 1 WBTC → 1 share.
3. Before the transaction is executed, the vault's share price increases (e.g., underlying assets reprice), so the exchange rate becomes 1 WBTC → 0.8 shares-equivalent, meaning shares are now more expensive.
4. When executed, the same 1 WBTC deposit mints fewer shares than expected (because each share now represents more value).

In this scenario, the operator may prefer to revert and retry later, but the modules provide no mechanism to enforce that.

Impact

- Deposits: the Superloop vault may mint fewer external-vault shares than expected for a given underlying amount.
- Withdrawals: the Superloop vault may burn more external-vault shares than expected to withdraw a given underlying amount.

This can lead to unfavorable execution for the strategy and makes operator actions vulnerable to adverse price movements between transaction creation and execution.

Code Snippet

https://github.com/sherlock-audit/2026-02-superlend-feb-12th-2026/blob/main/superloop-core-contracts/src/modules/vault/VaultSupplyModule.sol#L30
https://github.com/sherlock-audit/2026-02-superlend-feb-12th-2026/blob/main/superloop-core-contracts/src/modules/vault/VaultWithdrawModule.sol#L27

Tool Used

Manual Review

Recommendation

Add explicit slippage controls to both actions, for example:

- Supply: require `shares >= minSharesOut` (passed in params), reverting if fewer shares are minted than expected.
- Withdraw: either:
  - use `redeem(shares, ...)` with a `minAssetsOut` constraint, or
  - if sticking with `withdraw(assets, ...)`, compute/track an upper bound and require `sharesBurned <= maxSharesIn`.

This allows the operator to bound execution outcomes and revert transactions when the external vault's share price moves unfavorably.

Issue L-2: Superloop vault deviates from ERC-4626 specification [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-02-superlend-feb-12th-2026/issues/12

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

1) `maxDeposit` / `maxMint` can overestimate what `deposit` / `mint` will accept

ERC-4626 requires `maxDeposit(receiver)` and `maxMint(receiver)` to return a value that does not cause `deposit` / `mint` to revert (it may underestimate if necessary):

> maxDeposit
> MUST return the maximum amount of assets `deposit` would allow to be deposited for `receiver` and not cause a revert, which MUST NOT be higher than the actual maximum that would be accepted (it should underestimate if necessary). This assumes that the user has infinite assets, i.e. MUST NOT rely on balanceOf of asset.

However, `deposit` and `mint` enforce an additional constraint based on `_getCashReserveShortfall(totalAssetsCached)`:

```
uint256 cashReserveShortfall = _getCashReserveShortfall(totalAssetsCached);
require(assets <= cashReserveShortfall, Errors.INSUFFICIENT_CASH_SHORTFALL);
```

`maxDeposit` and `maxMint` currently only account for the supply cap, and ignore this cash reserve shortfall. Therefore, they can return values that are higher than what `deposit` / `mint` will accept, violating the ERC-4626 requirement that `max*` functions must not overestimate.

Note: The function `maxDeposit` is also used from `DepositManager`, so fixing this issue would require some refactoring on the `DepositManager` as well.

2) `maxWithdraw` / `maxRedeem` ignore "instant liquidity" constraints

`maxWithdraw(owner)` is expected to represent the maximum assets that can be withdrawn for `owner` without reverting via `withdraw`:

> maxWithdraw
> MUST return the maximum amount of assets that could be transferred from `owner` through `withdraw` and not cause a revert, which MUST NOT be higher than the actual maximum that would be accepted (it should underestimate if necessary).

Given that the Superloop vault has most of its assets deposited into external protocols like Aave, it means `maxWithdraw` must be bounded by the vault's immediately available asset balance.

Current default implementation:

```
function maxWithdraw(address owner) public view virtual returns (uint256) {
    return _convertToAssets(balanceOf(owner), Math.Rounding.Floor);
}
```

This value only depends on the owner's shares and conversion rate. It does not account for the vault's actual underlying asset balance or any other liquidity constraint. If the vault cannot satisfy full redemption instantly, `withdraw` may revert even though `assets <= maxWithdraw(owner)` passed, meaning `maxWithdraw` can overestimate.

The same issue applies to `maxRedeem(owner)`.

3) `withdraw` does not transfer the requested `assets` amount

ERC-4626 `withdraw(assets, receiver, owner)` is defined such that the caller requests a specific amount of `assets`, and the vault burns the necessary shares to transfer that exact `assets` amount to `receiver` (assuming no revert).

Current withdraw logic computes a fee in assets, reduces the amount actually redeemed, and sends less than `assets` to the receiver:

```
uint256 feeAssets = _getInstantWithdrawFeeValue(assets);
uint256 assetsToRedeem = assets - feeAssets;

// ...

_withdraw(_msgSender(), receiver, owner, assetsToRedeem, sharesToRedeem);
```

The fee is then charged by transferring additional shares to the treasury:

```
uint256 sharesToRedeem = previewWithdraw(assetsToRedeem);
uint256 sharesToTreasury = previewWithdraw(feeAssets);

// ...
if (sharesToTreasury > 0) {
    _transfer(owner, SuperloopStorage.getSuperloopEssentialRolesStorage().treasury,
    ↪  sharesToTreasury);
}
```

This causes the `receiver` to receive fewer assets than the specified `assets` amount.

OpenZeppelin's guidance for ERC-4626 fee support generally preserves the invariant that `withdraw(assets, ...)` delivers assets and accounts for fees through share

accounting (e.g., burning more shares / minting fee shares), rather than silently reducing the transferred assets.

Refer to the OZ docs for consultation: https://docs.openzeppelin.com/contracts/5.x/erc4626#custom-behavior-adding-fees-to-the-vault

4) `previewWithdraw` / `previewRedeem` do not include the instant withdrawal fee

Because `withdraw` charges an instant withdrawal fee, compliant previews should reflect the actual share/asset amounts that will be consumed/received according to the ERC-4626 specification:

> previewWithdraw
> MUST return as close to and no fewer than the exact amount of Vault shares that would be burned in a withdraw call in the same transaction. I.e. withdraw should return the same or fewer shares as previewWithdraw if called in the same transaction.
> MUST be inclusive of withdrawal fees. Integrators should be aware of the existence of withdrawal fees.

Current previews only include performance-fee conversion mechanics:

```
function previewWithdraw(uint256 assets) public view override returns (uint256) {
    uint256 totalAssetsCached = totalAssets();
    return _convertToSharesWithPerformanceFee(assets, totalAssetsCached,
    ↪  Math.Rounding.Ceil);
}

function previewRedeem(uint256 shares) public view override returns (uint256) {
    uint256 totalAssetsCached = totalAssets();
    return _convertToAssetsWithPerformanceFee(shares, totalAssetsCached,
    ↪  Math.Rounding.Floor);
}
```

They do not model the instant withdrawal fee that `withdraw` / `redeem` applies. This causes previews to misquote outcomes for integrators and users and breaks ERC-4626 compliance.

Impact

Some of the Superloop vault functions break the compatibility with ERC-4626, which can be an issue for external contracts that try to integrate with the Superloop vault.

Code Snippet

- https://github.com/sherlock-audit/2026-02-superlend-feb-12th-2026/blob/main/superloop-core-contracts/src/core/Superloop/SuperloopVault.sol#L39
- https://github.com/sherlock-audit/2026-02-superlend-feb-12th-2026/blob/main/superloop-core-contracts/src/core/Superloop/SuperloopVault.sol#L73
- https://github.com/sherlock-audit/2026-02-superlend-feb-12th-2026/blob/main/superloop-core-contracts/src/core/Superloop/SuperloopVault.sol#L63
- https://github.com/sherlock-audit/2026-02-superlend-feb-12th-2026/blob/main/superloop-core-contracts/src/core/Superloop/SuperloopVault.sol#L68
- https://github.com/sherlock-audit/2026-02-superlend-feb-12th-2026/blob/main/superloop-core-contracts/src/core/Superloop/SuperloopVault.sol#L123

Tool Used

Manual Review

Recommendation

Choose one of the following approaches:

1. Make the vault ERC-4626-compliant by fixing all the mentioned issues, which would require some refactoring to ensure the new changes don't affect other sections of the vault and related modules.
2. Explicitly document deviations from ERC-4626 so that future integrators and third-party tooling can be aware of them.

Issue L-3: Deposits may revert due to internal vault cap being reached [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-02-superlend-feb-12th-2026/issues/13

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Deposits may revert if the vault's cap is reached

Vulnerability Detail

When executing withdraws with `VaultWithdrawModule` we withdraw with `maxWithdraw`.

```
uint256 amount =
    params.amount == type(uint256).max ?
    ↪  IERC4626(params.vault).maxWithdraw(address(this)) : params.amount;
```

However when supplying with `VaultSupplyModule` we either do `balanceOf` to try and deposit 100% of our assets or use `amount` for a specific amount:

```
uint256 amount =
    params.amount == type(uint256).max ?
    ↪  IERC20(underlyingAsset).balanceOf(address(this)) : params.amount;
```

However the vault max supply cap may be bellow `IERC20(underlyingAsset).balanceOf(address(this))` and hard to reach by relying on `params.amount` due to other deposits/withdraws from other users constantly being executed.

This will result in TX reverting if we are trying to deposit more than `maxDeposit()` for that vault.

Impact

TX will fail if we are trying to deposit `IERC20(underlyingAsset).balanceOf(address(this))`, but that is more than `maxDeposit()` for that vault.

Tool Used

Manual Review

Recommendation

Consider checking for `maxDeposit()` before depositing the amount. Example code can be:

```
    function execute(DataTypes.VaultActionParams memory params) external override
    ↪  onlyExecutionContext {
        address underlyingAsset = IERC4626(params.vault).asset();

-       uint256 amount = params.amount == type(uint256).max ?
 ↪  IERC20(underlyingAsset).balanceOf(address(this)) : params.amount;

+       uint256 maxDeposit = IERC4626(params.vault).maxDeposit(address(this));
+       uint256 amount = params.amount == type(uint256).max ?
 ↪  IERC20(underlyingAsset).balanceOf(address(this)) : params.amount;
+       amount = amount > maxDeposit ? maxDeposit : amount;

        if (amount == 0) return;
```

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.